AI-ASSISTED CODING

ASSISGNMENT-19.3

NAME:V.SIDDARTHA
HALL:2303A52062
BATCH:45

Task 1 – Python to C++ Conversion
Task:
Translate a simple Python class into C++ using AI assistance.
Instructions:
• Prompt AI to convert a Python class representing a Student
with attributes and methods into equivalent C++ code
• Ensure proper handling of:
o Constructors
o Data types
o Access specifiers
• Compile and run both versions to verify output consistency
Expected Output:
An equivalent working C++ class translated from Python.

#PROMPT
generate code of any python class representing a Student with attributes and methods with proper usage of Constructors,Data types, Access specifiers.

#PYTHON CODE

```
class Student:
    def __init__(self, name, age, grade):
        self.name = name  # Public attribute
        self._age = age   # Protected attribute
        self.__grade = grade  # Private attribute

    def get_grade(self):
        return self.__grade

    def set_grade(self, grade):
        if 0 <= grade <= 100:
            self.__grade = grade
        else:
            print("Invalid grade. Must be between 0 and 100.")
```

```
    def display_info(self):
        print(f"Name: {self.name}")
        print(f"Age: {self._age}")
        print(f"Grade: {self.__grade}")
# Example usage
if __name__ == "__main__":
    student1 = Student("Siddhu", 20, 85)
    student1.display_info()
    student1.set_grade(90)
    print("Updated Grade:", student1.get_grade())
```

#OUTPUT

#PYTHONCODE CONVERTED INTO C++

```
#include <iostream>
#include <string>
using namespace std;
class Student {
private:
    string name; // Private attribute
    int age;     // Private attribute
    int grade;   // Private attribute
public:
    // Constructor
    Student(string n, int a, int g) : name(n), age(a), grade(g) {}
    // Getter for grade
    int getGrade() {
        return grade;
    }
    // Setter for grade
    void setGrade(int g) {
        if (g >= 0 && g <= 100) {
            grade = g;
```

```
        } else {
            cout << "Invalid grade. Must be between 0 and 100." << endl;
        }
    }
    // Method to display student information
    void displayInfo() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "Grade: " << grade << endl;
    }
};
int main() {
    Student student1("Siddhu", 20, 85);
    student1.displayInfo();
    student1.setGrade(90);
    cout << "Updated Grade: " << student1.getGrade() << endl;
    return 0;
}
```

Task 2 – Java to Python Function Conversion

Task:

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- **Ask AI to translate a Java isPrime() method into Python**
- **Test the function with multiple inputs**
- **Ensure the Python version follows Pythonic syntax and logic**

Expected Output:

A correct Python function equivalent to the original Java prime-checking method.

#PROMPT

generate a java method that checks whether a given number is prime or not.Test the function with multiple inputs

#java code

```
public class PrimeChecker {
    public static boolean isPrime(int number) {
        if (number <= 1) {
            return false;
        }
```

```
        for (int i = 2; i <= Math.sqrt(number); i++) {
            if (number % i == 0) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        int[] testNumbers = {1, 2, 3, 4, 5, 10, 11, 13, 15, 17};
        for (int num : testNumbers) {
            System.out.println(num + " is prime: " + isPrime(num));
        }
    }
}
```

#OUTPUT

#JAVA CODE CONVERTED TO PYTHON

```
def is_prime(number):
    if number <= 1:
        return False
    for i in range(2, int(number**0.5) + 1):
        if number % i == 0:
            return False
    return True
test_numbers = [1, 2, 3, 4, 5, 10, 11, 13, 15, 17]
for num in test_numbers:
```

```
    print(f"{num} is prime: {is_prime(num)}")
```

Task 3 – Pseudocode to Python Implementation
Task:
Translate a given pseudocode algorithm into Python using AI.
Instructions:
- **Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)**
- **Ask AI to convert it into executable Python code**
- **Validate correctness using sample input lists**

Expected Output:
A working Python program that correctly implements the pseudocode logic.

#prompt
generate Pseudocode for sortinf=g numbers by using bubble sort algorithm and later convert it in to python code

#pseudocode
1. Start
2. Define a function bubble_sort that takes a list of numbers as input
3. Get the length of the list and store it in a variable n
4. Loop through the list from 0 to n-1
   a. For each element, loop through the list from 0 to n-i-1
      i. If the current element is greater than the next element, swap them
5. Return the sorted list
6. End

#converted python code
```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
# Example usage
if __name__ == "__main__":
    numbers = [64, 34, 25, 12, 22, 11, 90]
    sorted_numbers = bubble_sort(numbers)
    print("Sorted array is:", sorted_number
```

#output

```
# Example usage
if __name__ == "__main__":
    numbers = [64, 34, 25, 12, 22, 11, 90]
```

PROBLEMS 47 DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppDat
a\Local\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI
 - Assisted coding/.github/task3.py"
Sorted array is: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

powershell
Python
Python
Python
Build Workspace
Run: PrimeChecker

Task 4 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- **Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).**
- **Ask AI to generate equivalent Pandas code.**
- **Test the generated code on a sample DataFrame.**

Expected Output:

- **Equivalent Pandas query that retrieves the correct subset of data.**

#prompt

generate code of sql query with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).and later convert to equilanet pandas DataFrame operation in python

#sql query

SELECT name, salary FROM employees WHERE salary > 50000;

 # Pandas DataFrame operation in Python.

```
import pandas as pd
# Assuming 'employees' is a DataFrame containing employee data
# Example DataFrame
data = {
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'salary': [60000, 45000, 70000, 30000]
```

```
}
employees = pd.DataFrame(data)
# Pandas operation to filter employees with salary greater than 50000
filtered_employees = employees[employees['salary'] > 50000][['name', 'salary']]
print(filtered_employees)
```

Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

- **Converting a Python searching algorithm into C**
- **Translating a Java sorting program into JavaScript**

Instructions:

- **Use AI to translate a selected algorithm between two programming languages**
- **Execute and test both versions**
- **Document translation challenges such as:**
  - **Syntax differences**
  - **Library support**
  - **Memory management**

Expected Output:

Equivalent, tested code implementations in two different programming languages.

#prompt